

• ADR-LOCKED · IMPLEMENTATION-READY

PreOne Frontend Blueprint.

Phase 0 → Phase 6 · 20-week build plan
12 shared packages · ~200 module pages

Enterprise-grade monorepo architecture for a multi-tenant Pre-K & school management platform. Covers foundation, design system, auth/RBAC, OpenAPI codegen, all 14 functional modules, white-label Design Studio, and full CI/observing/deploy pipeline. Mobile (React Native) reuse baked into 7 of 12 packages. Prepared for AI implementation agent + engineering team handoff.

12

PACKAGES

20_{wk}

TIMELINE

~22.5_K

LOC SHARED

7/12

RN-READY

Document type: Frontend Engineering Spec
Stack: Next.js 16 · Turborepo · TypeScript strict
Status: Final · ADR-locked · Phase 0 ready

Table of Contents

A 16-chapter build plan for the PreOne multi-tenant Pre-K & school management platform. Covers foundation, design system, auth/RBAC, API layer, all 14 functional modules, white-label Design Studio, and full CI/performance/deploy pipeline.

1. Executive Summary5

Scope at a Glance5

Critical Path5

Key Architectural Decisions5

Project Maturity Status6

How to Use This Document6

2. Quick Index — 12 Packages at a Glance8

Dependency Matrix (Locked)8

Mobile-Compatibility Strategy9

3. Phase 0 — Enterprise Foundation (~1.5 weeks)10

Goal10

Packages to Create10

@preone/config — shared linting + formatting + versioning10

@preone/env — Zod-validated typed environment variables10

@preone/design-tokens — Style Dictionary source (75+ tokens, Phase 0 lockable set frozen) . 11

@preone/storybook — shared Storybook 8 config11

@preone/core — merged package (logger + analytics + telemetry + errors + constants) 11

Root-Level Monorepo Files12

Tasks12

Acceptance Criteria12

4. Phase 1 — Design System (~3 weeks)14

Goal14

Packages to Create14

@preone/ui — ShadCN v2 primitives + PreOne composites14

@preone/forms — React Hook Form wrappers (separate from UI)15

@preone/tables — TanStack Table v8 (separate ecosystem)	16
@preone/charts — Recharts, theme-aware	16
@preone/notifications — toast + realtime + push	17
Theme Engine Implementation	17
Tasks per Component	18
Acceptance Criteria	18
5. Phase 2 — Authentication + Layout + RBAC (~2 weeks)	20
Goal	20
@preone/auth — shared web + mobile	20
Auth Flows	21
8 Layouts	21
7-Layer RBAC Implementation	22
Multi-Tenant Context Switcher	22
Tasks	22
Acceptance Criteria	23
6. Phase 3 — API Layer + Query + i18n (~1.5 weeks)	24
Goal	24
OpenAPI Codegen Pipeline	24
Codegen Flow	25
12 Service Files	25
TanStack Query Setup	26
Zustand Store	26
i18n (next-intl v3)	27
Tasks	27
Acceptance Criteria	28
7. Phase 4 — Modules (P1 → P5) (~6 weeks)	29
Goal	29
Locked Feature Folder Structure	29
Module Priority + Page Counts	29
Per-Module Deliverables	30
Sample Module Breakdown — M04 Student (13 pages)	31

8. Phase 5 — Design Studio (~4 weeks)	32
Goal	32
Feature Folder Structure	32
13 Customization Features	32
Workflow (per UI Phil §3.3)	33
Access Matrix Enforcement	33
Backend Dependencies	34
Acceptance Criteria	34
9. Phase 6 — Testing + Performance + Deploy (~2 weeks)	36
Goal	36
Expanded Testing Matrix	36
CI Pipeline (Final, Expanded)	37
Performance Budget Enforcement	37
Performance Tactics Applied	38
Observability Stack	38
Deployment Topology	39
Acceptance Criteria	39
10. Final Timeline & Critical Path	41
Phase-by-Phase Timeline	41
Critical Path	41
Parallelization Opportunities	41
11. Risk Areas & Mitigations	43
12. Refinement Integration Log (All 15 + 4 + 3 Modifications)	44
Round 1 — Original 15 Modifications	44
Round 2 — Four Refinements (Package Consolidation)	45
Refinement 1 — Package Count Freeze: @preone/core merge	45
Refinement 2 — No Module Federation: Monorepo Only	45
Refinement 3 — Design System Lock: Foundation Tokens Frozen in Phase 0	45
Refinement 4 — Mobile Reuse: 7 of 12 packages RN-compatible	45
Round 3 — Final Three Architectural Locks	45

13. @preone/core — Merged Package Deep Dive	47
Rationale	47
Internal File Structure	47
Cross-Concern Coupling Map	48
Pure TS — Works in Web and RN	49
14. Design System Lock — Frozen Tokens & Enforcement	50
Tokens Frozen in Phase 0	50
Three-Layer Lock Enforcement	50
Layer 1 — ESLint Rule (automated)	50
Layer 2 — Token Change Governance (manual)	51
Layer 3 — Visual Regression (automated)	51
Phase 0 Deliverable — Frozen tokens.json	51
15. Mobile Reuse Strategy — Cross-Platform Compatibility	53
Per-Package Compatibility Matrix	53
Three Mobile-Compatible Package Patterns	53
Pattern 1 — Pure TS (no platform code)	53
Pattern 2 — Split export with platform-specific entry points	54
Pattern 3 — Core + Adapters	55
What This Enables	55
16. Glossary & AI Implementation Prompt	57
Glossary	57
AI Implementation Prompt	58
Phase Completion Checklist (Per Phase)	59

CHAPTER 1

1. Executive Summary

This document is the **implementation-ready frontend blueprint** for PreOne — a multi-tenant Pre-K and school management platform. It consolidates seven iterative planning rounds (Phase 0 through Phase 6) into a single ADR-locked specification that an AI implementation agent or a human engineering team can execute without further design decisions. Every package, file tree, acceptance criterion, and architectural rule has been frozen by an Architecture Decision Record (ADR) and cannot be changed without explicit ADR approval.

Scope at a Glance

The blueprint covers 12 shared packages across 7 sequential phases, totalling approximately **22,500 lines of shared TypeScript code** and roughly **200 module pages** across 14 functional modules. The end-to-end timeline is **20 weeks (5 months)** on the critical path, with parallelization opportunities that allow Phase 5 (Design Studio) and parts of Phase 4 (P2 modules) to overlap with prior-phase polish work. The platform supports three locales (en-IN, hi, mr), three client apps (web, parent-app, teacher-app), and a fourth planned React Native mobile build that reuses approximately 60% of the shared codebase.

Critical Path

The critical path is strictly sequential: **Phase 0 (Enterprise Foundation) → Phase 1 (Design System) → Phase 2 (Auth + Layout + RBAC) → Phase 3 (API Layer + Query + i18n) → Phase 4 (Modules P1-P5) → Phase 5 (Design Studio) → Phase 6 (Testing + Performance + Deploy)**. Each phase produces a stable, contract-tested foundation that the next phase depends on. Skipping or reordering phases is forbidden by the ADR. Phase 6 testing runs continuously from Phase 1 onwards; the final two weeks are reserved exclusively for hardening and performance tuning rather than net-new test authoring.

Key Architectural Decisions

- **Monorepo only — no Module Federation.** A single Next.js app in apps/web/ orchestrated by Turborepo. Cold build ~3-5 min, incremental with Turbo cache ~30-60s. MFE is revisited only if build time exceeds 10 min in future.
- **@preone/core merged package.** The original 14-package plan was consolidated to 12 by merging logger, analytics, telemetry, errors, and constants into a single @preone/core package. These five concerns are tightly coupled (errors need traceId from telemetry; logger decides error level; event names from constants feed analytics) and merging them prevents circular dependencies.
- **Design tokens frozen in Phase 0.** Spacing, typography, elevation, motion, radius, and focus-ring tokens are locked in tokens.json at Phase 0 sign-off. Changes require an ADR + Design Council approval + semver major bump + migration codemod. Color tokens are *not* frozen — they are per-tenant via the

Theme Engine.

- **7 of 12 packages are React Native compatible.** core, env, design-tokens, ui (split export), forms (split export), auth, api-client, and notifications are designed to work in both web and RN. tables and charts are web-only (RN uses FlashList and victory-native respectively). Estimated mobile reuse: ~60% shared, ~40% RN-specific.
- **100% OpenAPI codegen — no manual Axios.** Backend NestJS generates openapi.json committed to repo; openapi-typescript + openapi-fetch generate the typed client (~3KB gzipped vs ~30KB for Axios). Contract test in CI fails any PR where the spec drifts from the committed version.
- **ADR-locked architecture.** Package structure, dependency matrix, feature folder layout, naming convention, and import convention are all declared as ADR-locked. Feature packages may *never* depend on each other — only on shared packages.

Project Maturity Status

All upstream artefacts required to begin frontend coding are complete and signed off. The blueprint therefore marks the transition from planning to implementation.

Artefact	Status	Notes
Vision	Signed-off	Product vision & long-term roadmap
BRC (Business Requirements)	Signed-off	Cross-functional business contract
PRD (Product Requirements)	Signed-off	Detailed functional requirements
DDD (Domain-Driven Design)	Signed-off	Bounded contexts identified
ADR (Architecture Decisions)	Locked	Package structure, deps, naming frozen
ERD (Entity-Relationship)	Signed-off	Database schema across 14 modules
API Contracts (OpenAPI)	Signed-off	Backend Swagger committed
RBAC Matrix	Signed-off	7-layer RBAC fully specified
UI Philosophy	Signed-off	Fluent Metro design language
Backend	Signed-off	NestJS + Postgres + Redis
Backend Hardening	Signed-off	Security, rate-limit, observability
Frontend Blueprint	Locked	This document (ADR-locked)
Enterprise Architecture	Signed-off	Multi-tenant + white-label strategy

Table 1.1 — Upstream artefacts. All [signed-off]. Implementation is the next stage.

How to Use This Document

For an AI implementation agent: read this entire document, then begin with Chapter 3 (Phase 0) and proceed sequentially through Chapter 9 (Phase 6). For each phase, implement every listed package with the exact file structure shown, satisfy all acceptance criteria, and run all listed CI gates before proceeding to the next phase. Append a work record to worklog.md after each phase. The implementation prompt at the end of Chapter 16 is the copy-paste-ready instruction for the build agent. For human readers: Chapters 2-9 are the phase-by-phase build plan; Chapters 10-15 are reference material (timeline, risks, refinement log, core deep-dive, design lock, mobile strategy); Chapter 16 is a glossary plus the AI handoff prompt.

CHAPTER 2

2. Quick Index — 12 Packages at a Glance

PreOne is structured as a pnpm + Turborepo monorepo with 12 shared packages under packages/ and four client apps under apps/. The table below is the master index — every package, its introducing phase, mobile-compatibility status, and rough line-of-code estimate. Total shared code: approximately 22,500 LOC.

#	Package	Phase	Mobile	LOC est.	Purpose
1	@preone/config	0	✓	~500	ESLint + TSConfig + Prettier + Commitlint + Changesets
2	@preone/env	0	✓	~300	Zod-validated typed env parser per app
3	@preone/design-tokens	0	✓	~800	Style Dictionary source (75+ tokens, locked)
4	@preone/storybook	0	✗	~200	Shared Storybook 8 config (web only)
5	@preone/core	0	✓	~1500	Merged: logger + analytics + telemetry + errors + constants
6	@preone/ui	1	split	~8000	ShadCN primitives + PreOne composites (40+ components)
7	@preone/forms	1	split	~2000	RHF + Zod (logic shared, rendering split)
8	@preone/tables	1	✗	~3000	TanStack Table v8 (web only — RN uses FlashList)
9	@preone/charts	1	✗	~1500	Recharts theme-aware (web only — RN uses victory-native)
10	@preone/notifications	1	✓	~1200	Toast + Realtime (shared) + Push (platform-specific)
11	@preone/auth	2	✓	~2500	JWT + Session + RBAC (fully shared web+RN)
12	@preone/api-client	3	✓	~1000	OpenAPI codegen + fetch wrapper (mostly auto-generated)

Table 2.1 — 12 shared packages. ✓ = RN-ready, ✗ = web-only, split = web/native dual export.

Dependency Matrix (Locked)

Package dependencies flow strictly downward through the matrix below. A package may only depend on packages listed above it. Feature packages (under apps/web/src/features/*) may depend on any shared package but must **never** depend on each other — cross-feature imports are forbidden by ESLint rule import-boundaries and fail CI.

DEPENDENCY MATRIX

```

config
↓
env
↓
core
↓
design-tokens
↓
ui
↓
forms
↓
auth
↓
api-client
↓
features/* (student, finance, attendance, admissions, ...)

```

Mobile-Compatibility Strategy

Each package falls into one of three mobile-compatibility patterns. The pattern determines how the package is structured and how the React Native parent-app (apps/mobile/ — PreOne Hub) consumes it. Full strategy details are in Chapter 15.

Pattern	Used by	Strategy
Pure TS (no platform code)	config, env, design-tokens, core, auth (core), api-client	Single source — works in both web and RN with no platform branching
Split export (web + native)	ui, forms	shared/ (types, variants) + web/ (DOM impl) + native/ (RN impl). package.json exports map picks one at build time
Core + Adapters	auth (token storage), notifications (push)	Interface in core; platform-specific adapter implementations injected at create time
Web only	storybook, tables, charts	Web-only by design — RN equivalents (FlashList, victory-native, Storybook RN) used in mobile app

Table 2.2 — Four package patterns for cross-platform reuse.

CHAPTER 3

3. Phase 0 — Enterprise Foundation (~1.5 weeks)

Goal

Production-grade monorepo skeleton with all shared config, design tokens, Storybook, and the merged `@preone/core` package — ready for any subsequent package to be built on top. By end of Phase 0, an engineer can run `pnpm install`, `pnpm turbo build`, `pnpm turbo lint`, `pnpm turbo typecheck`, and `pnpm storybook` with zero errors. The phase also locks the design token foundation (spacing, typography, elevation, motion, radius, focus ring) — these tokens are FROZEN after sign-off and changes require an ADR.

Packages to Create

@preone/config — shared linting + formatting + versioning

A meta-package containing five sub-packages that govern all code style, type checking, commit conventions, and versioning across the monorepo.

Sub-package	Files	Purpose
eslint-config/	base.js, next.js, react-lib.js, node.js	Flat config v9. Rules: import-boundaries, jsx-ally, react-hooks, no-any, no-console, no-raw-design-values (custom rule blocking raw padding/color/radius values)
tsconfig/	base.json, nextjs.json, react-lib.json, node.json	Project references + path aliases (<code>@preone/*</code>). Strict mode enabled across all configs
prettier-config/	index.js	Shared format rules: printWidth 100, semi true, single quote, trailing comma all
commitlint-config/	index.js	Conventional commits (feat / fix / chore / refactor / docs / breaking). Scopes per package enforce monorepo hygiene
changeset-config/	config.json	Versioning + changelog per package. <code>pnpm changeset</code> generates changelog entries; <code>pnpm release</code> publishes

Table 3.1 — @preone/config sub-packages

@preone/env — Zod-validated typed environment variables

Each app (web, mobile, worker) calls `loadEnv(preset)` at boot and receives a fully-typed `process.env` object that has been runtime-validated against a Zod schema. Missing or malformed env vars fail fast at startup rather than causing silent runtime errors deep in the call stack.

text

```
packages/env/
├── src/
│   ├── env.ts # Zod-validated typed env parser
│   ├── presets/
│   ├── web.ts # NEXT_PUBLIC_API_URL, NEXT_PUBLIC_SENTRY_DSN, ...
│   ├── mobile.ts # EXPO_PUBLIC_API_URL, EXPO_PUBLIC_SENTRY_DSN, ...
│   ├── worker.ts # REDIS_URL, DATABASE_URL, SENTRY_DSN, ...
│   └── index.ts
```

@preone/design-tokens — Style Dictionary source (75+ tokens, Phase 0 lockable set frozen)

The design-tokens package is the single source of truth for all visual values in the platform. It uses Style Dictionary to compile a single tokens.json into CSS variables, TypeScript constants, JS objects, and Figma-compatible JSON. Token layers: **Raw** (primitive values like color-purple-500: #6A35FF), **Semantic** (aliases like --primary: {color.purple.500}), **Component** (optional, like --button-primary-bg: {primary}), and **Theme overrides** (per-tenant JSON that replaces semantic mappings).

text

```
packages/design-tokens/
├── tokens.json # Style Dictionary source (raw tokens)
├── transforms/ # custom transforms
│   ├── color-oklch.ts # color/oklch
│   └── spacing-rem.ts # spacing/rem
├── build/ # generated outputs
│   ├── css/tokens.css
│   ├── js/tokens.ts
│   ├── ts/tokens.d.ts
│   └── figma/tokens.json
└── tailwind-theme.css # @theme block for Tailwind v4
```

@preone/storybook — shared Storybook 8 config

A single shared Storybook config that all packages consume — no per-package Storybook setup. This eliminates drift between component development environments and ensures every package sees the same decorators, addons, and theme configuration.

text

```
packages/storybook/
├── main.ts # addons: a11y, controls, actions, viewport, themes, docs
├── preview.tsx # global decorators: ThemeProvider, I18nProvider, MockAuth
└── manager.ts # custom theme matching PreOne Fluent Metro
```

@preone/core — merged package (logger + analytics + telemetry + errors + constants)

Originally five separate packages, consolidated into one to eliminate circular dependencies. The five concerns are tightly coupled: errors carry a traceId generated by telemetry; the logger decides what level an error gets logged at; analytics events are named by constants; constants also feed RBAC codes consumed by errors. Full

internal structure and rationale are detailed in Chapter 13.

Root-Level Monorepo Files

text

```
preone/
├─ pnpm-workspace.yaml
├─ turbo.json # build, lint, typecheck, test, storybook, dev — with cache
├─ package.json # root scripts + devDeps
├─ .husky/
│   └─ pre-commit # lint-staged
│       └─ commit-msg # commitlint
├─ .changeset/
│   └─ config.json
├─ .github/workflows/
│   └─ ci.yml # lint → typecheck → unit → component → a11y → build
│       └─ release.yml # changesets publish
├─ apps/
├─ web/ # Next.js 16 dashboard
├─ mobile/ # PreOne Hub (React Native parent app)
├─ parent-app/ # parent-facing web app
└─ teacher-app/ # teacher-facing web app
```

Tasks

- Init pnpm workspace + turbo 2.x
- Create all packages/config/* sub-packages
- Create packages/env/ with Zod validation
- Create packages/design-tokens/ with Style Dictionary + 75+ tokens from UI Phil Appendix A
- Setup packages/storybook/ shared config (Storybook 8)
- Create packages/core/ merged package (logger + analytics + telemetry + errors + constants)
- Configure Husky pre-commit (lint-staged: eslint --fix + prettier --write + tsc --noEmit on staged files)
- Configure commitlint (conventional commits, scopes per package)
- Setup changesets for versioning
- GitHub Actions CI: matrix job for lint + typecheck + unit + a11y on every PR
- Path aliases: @preone/ui, @preone/auth, @preone/forms, @preone/tables, @preone/charts, @preone/api-client, @preone/core, @preone/notifications, @preone/env, @preone/design-tokens
- ESLint rule for design-token enforcement (no raw values for spacing/color/radius/shadow/motion)

Acceptance Criteria

- ✓ pnpm install works (no errors)

- ✓ pnpm turbo build builds all packages
- ✓ pnpm turbo lint passes (0 errors, 0 warnings)
- ✓ pnpm turbo typecheck passes
- ✓ pnpm storybook starts with empty stories list
- ✓ Non-conventional commit blocked by commitlint
- ✓ Pre-commit hook runs lint-staged
- ✓ CI green on empty repo
- ✓ pnpm changeset generates changelog entry
- ✓ Design tokens build to CSS / JS / TS formats

INFO

Time estimate: ~1.5 weeks. **Phase 0 deliverable is the locked tokens.json file** — once signed off, structural tokens (spacing, typography, elevation, motion, radius, focus ring) cannot change without an ADR + Design Council approval. See Chapter 14 for the full frozen token values.

CHAPTER 4

4. Phase 1 — Design System (~3 weeks)

Goal

All 75+ design tokens live as CSS variables, the 4-level theme cascade is functional (Platform → Subscription → School → Branch → User), 40+ base components ship with Storybook stories for every variant and state, and light/dark/auto theme modes work end-to-end. By end of Phase 1, the design system is the only sanctioned way to build UI — raw Tailwind utility classes are blocked by ESLint unless they map to a design token.

Packages to Create

@preone/ui — ShadCN v2 primitives + PreOne composites

The UI package is split into two layers. **Primitives** are ShadCN-based components that we own and maintain (forked, not consumed as a dependency). **Composites** are PreOne-specific combinations of primitives that encode our domain patterns — dashboard tiles, page headers, sidebars with RBAC-filtered nav, etc.

text

```

packages/ui/
├── src/
│   ├── primitives/ # ShadCN-based, owned
│   │   ├── button/ # variants: primary, secondary, ghost, danger, link
│   │   │   # sizes: sm / md / lg
│   │   ├── input/ # text, password, number, email, search
│   │   │   # prefix / suffix icons
│   │   ├── select/ # single / multi / async, searchable, grouping
│   │   ├── textarea/ # auto-resize, char count, max length
│   │   ├── checkbox/ # indeterminate, group, checkAll
│   │   ├── radio/ # group with cards, h/v layout
│   │   ├── switch/ # toggle, loading, disabled
│   │   ├── date-picker/ # single / range, time, locale-aware, shortcuts
│   │   ├── badge/ # count, dot, label; colors per status
│   │   ├── avatar/ # image / initials, sizes, group overlap
│   │   ├── tooltip/ # hover / focus, positions
│   │   ├── popover/ # rich tooltip, interactive
│   │   ├── dialog/ # modal, sizes sm / md / lg / xl, ESC close
│   │   ├── drawer/ # side panel, left / right, nested
│   │   ├── dropdown-menu/ # kebab menu, items
│   │   ├── separator/
│   │   ├── scroll-area/
│   │   ├── tabs/ # underline / pills / cards variants
│   │   ├── accordion/ # collapsible sections
│   │   ├── progress/ # horizontal, color variants
│   │   ├── skeleton/ # shimmer animation, text / card / row variants
│   │   └── sonner/ # toast primitive (sonner wrapper)
│   ├── composites/ # PreOne-specific
│   │   ├── card/ # header / body / footer, hoverable, selectable
│   │   ├── tile/ # dashboard metric: number + label + trend + view-link
│   │   ├── page-header/ # title + subtitle + breadcrumb + actions
│   │   ├── sidebar/ # nav tree, RBAC-filtered, collapse modes
│   │   ├── header/ # logo + search + notifications + profile
│   │   ├── bottom-nav/ # mobile 5 primary + More
│   │   ├── form-layout/ # responsive grid wrapper
│   │   ├── breadcrumb/ # auto from route + custom items
│   │   ├── empty-state/ # illustration + headline + subtext + CTA
│   │   ├── error-state/ # illustration + retry
│   │   ├── confirm-dialog/ # destructive action confirmation
│   │   └── data-table-shell/ # visual shell (logic in @preone/tables)
│   ├── stories/ # .stories.tsx per component (CSF 3.0)
│   └── index.ts # barrel export

```

@preone/forms — React Hook Form wrappers (separate from UI)

Forms are deliberately kept separate from UI so the validation logic (RHF + Zod) can be reused in React Native without dragging DOM-specific rendering along. The package is split into a pure-logic core (Zod schemas, resolvers) and platform-specific rendering layers (web DOM vs RN TextInput).

text


```

packages/forms/
├── src/
│   ├── form.tsx # RHF FormProvider wrapper
│   ├── form-field.tsx # label + input slot + error + helper text
│   ├── form-section.tsx # collapsible section for long forms
│   ├── form-array.tsx # field arrays (guardians, items)
│   ├── form-submit.tsx # submit button with loading state
│   ├── validators/ # reusable Zod schemas
│   │   ├── phone.ts # +91 10-digit [6-9]\d{9}
│   │   ├── email.ts # RFC 5322 + lowercase
│   │   ├── aadhaar.ts # 12-digit + Verhoeff checksum
│   │   ├── pan.ts # ABCDE1234F
│   │   ├── dob-preschool.ts # age 2-6
│   │   ├── fee-amount.ts # paise, min 1, max 10000000
│   │   ├── date-range.ts # end >= start, max 365 days
│   │   └── zod-resolver.ts # RHF + zod bridge
│   └── index.ts

```

@preone/tables — TanStack Table v8 (separate ecosystem)

Tables are split into their own package because TanStack Table is a headless library with its own complex state model — bundling it inside @preone/ui would violate the bundle-size budget for primitive components. The package provides a styled shell on top of headless logic, plus built-in filters, pagination, sorting, virtualization for 1000+ rows, and Excel/CSV/PDF export.

text

```

packages/tables/
├── src/
│   ├── data-table.tsx # main wrapper (headless logic + styled shell)
│   ├── columns/ # column types (text, number, date, badge, action)
│   ├── filters/
│   │   ├── text-filter.tsx
│   │   ├── select-filter.tsx
│   │   ├── date-range-filter.tsx
│   │   └── multi-select-filter.tsx
│   ├── pagination.tsx # cursor + page-based
│   ├── column-visibility.tsx # user can hide / show, persisted
│   ├── export.tsx # Excel / CSV / PDF trigger
│   ├── density.tsx # compact / regular / comfortable
│   ├── selection.tsx # row selection + bulk action bar
│   ├── sorting.tsx # single + multi-column
│   ├── virtualization.tsx # TanStack Virtual for 1000+ rows
│   ├── types.ts # ColumnDef extensions, TableState
│   └── index.ts

```

@preone/charts — Recharts, theme-aware

All chart components read CSS custom properties (--primary, --success, etc.) so they re-theme instantly when the tenant changes. Nine chart types cover all platform visualizations — line, bar (stacked + grouped), pie (donut variant), area (stacked), scatter, heatmap, funnel, gauge, and inline sparkline. Shared tooltip, legend, responsive-container, and empty-state components ensure visual consistency.

text

```

packages/charts/
├── src/
│   ├── line-chart.tsx
│   ├── bar-chart.tsx # stacked + grouped
│   ├── pie-chart.tsx # donut variant
│   ├── area-chart.tsx # stacked
│   ├── scatter-chart.tsx
│   ├── heatmap-chart.tsx
│   ├── funnel-chart.tsx
│   ├── gauge-chart.tsx
│   ├── sparkline.tsx # inline mini-chart
│   └── shared/
│       ├── tooltip.tsx # theme-aware tooltip
│       ├── legend.tsx # below chart, collapsible
│       ├── responsive-container.tsx
│       └── empty-state.tsx # when no data
├── tokens.ts # reads --primary, --success etc.
└── index.ts

```

@preone/notifications — toast + realtime + push

Three notification channels in one package. **Toast** wraps Sonner for ephemeral in-app messages. **Realtime** is a Socket.IO v4 client with React hooks for subscribing to events. **Push** uses the Web Push API on web and platform-native push on RN — the shared interface means consumers do not care which channel delivered the notification.

text

```

packages/notifications/
├── src/
│   ├── toast/
│   │   ├── toast-provider.tsx # sonner wrapper
│   │   ├── toast.ts # API: success/error/warn/info/loading/promise
│   │   └── action-toast.tsx # with action button
│   ├── realtime/
│   │   ├── socket-provider.tsx # Socket.IO v4 client
│   │   ├── use-realtime.ts # hook: subscribe to event
│   │   └── rooms.ts # join / leave room helpers
│   └── push/
│       ├── push-manager.ts # Web Push API
│       ├── permission.ts # request / check
│       └── service-worker.ts # SW registration + push handler
└── index.ts

```

Theme Engine Implementation

The Theme Engine is the heart of PreOne's multi-tenant white-label capability. It resolves a 4-level cascade (Platform → Subscription → School → Branch → User) into a single set of CSS variables injected on :root. Switching schools does not trigger a JS re-render — only CSS variables change, and the browser repaints. This is critical for performance: a React re-render of the entire tree on every school switch would be janky.

text

```
packages/ui/src/theme/
├─ theme-provider.tsx # resolves cascade, injects CSS vars on :root
├─ use-theme.ts # read current theme, set user preference
├─ theme-context.ts # React context
├─ cascade.ts # Platform → Subscription → School → Branch → User
├─ theme-schema.ts # TypeScript type matching DB JSONB
├─ theme-modes.ts # light / dark / auto / high-contrast / white-label
├─ theme-fetch.ts # GET /v1/theme with Redis cache
```

Tasks per Component

- Implement with TS strict types (no any)
- Accept className prop for Tailwind composition
- Forward ref via React.forwardRef
- ARIA labels + keyboard navigation
- Storybook story (CSF 3.0) — all variants + states + edge cases
- axe-core a11y test (vite-test-axe)
- Unit test (Vitest + Testing Library) for behavior
- Snapshot test for stable rendering
- JSDoc on exported props
- Add to barrel index.ts

Acceptance Criteria

- ✓ 40+ Phase-1 components shipped (per UI Phil §8.2)
- ✓ All 75+ design tokens live as CSS variables
- ✓ Theme cascade (4-level) functional — switch school → UI re-themes via CSS only
- ✓ Light / Dark / Auto mode toggle works
- ✓ All components pass axe-core (zero violations)
- ✓ All components have Storybook stories
- ✓ pnpm storybook shows all components organized by category
- ✓ pnpm test passes all unit + a11y + snapshot tests
- ✓ pnpm build produces per-package dist (ESM + CJS + dts)
- ✓ Bundle size: each primitive <5KB, composite <15KB gzipped

WARNING

Time estimate: ~3 weeks. The biggest schedule risk is the 4-level theme cascade — verify early (in week 1 of Phase 1) that switching school triggers **CSS-only** re-theming, not a React re-render. If a re-render occurs, the cascade implementation is broken.

CHAPTER 5

5. Phase 2 — Authentication + Layout + RBAC (~2 weeks)

Goal

Complete authentication flows (Phone OTP, Email Password, MFA, onboarding, session timeout, account locked), 8 distinct layouts, the 7-layer RBAC enforcement system, and the multi-tenant context switcher (School / Branch / Academic Year). By end of Phase 2, a user can log in, have their permissions resolved, navigate a RBAC-filtered sidebar, switch schools, and have the entire UI re-theme + re-permission without a page reload. The @preone/auth package is designed RN-compatible from day 1 — no web-only APIs in the core; only in adapters.

@preone/auth — shared web + mobile

text

```
packages/auth/
├── src/
│   ├── hooks/
│   │   ├── use-auth.ts # current user, login state
│   │   ├── use-session.ts # session lifecycle, idle timeout
│   │   └── use-permissions.ts # resolve + cache permissions
│   ├── providers/
│   │   ├── auth-provider.tsx # root context (user, tokens, status)
│   │   └── session-provider.tsx # session state, idle timer
│   ├── permissions/
│   │   ├── can.tsx # memoized component
│   │   │   # (permission / role / anyOf / allOf)
│   │   ├── permission-guard.tsx # screen-level guard with fallback UI
│   │   └── use-permission.ts # programmatic permission check hook
│   ├── session/
│   │   ├── token-store.ts # in-memory access token (Zustand slice)
│   │   ├── refresh-manager.ts # silent refresh orchestrator
│   │   ├── idle-timer.ts # 30-min idle → 25-min warning → logout
│   │   └── concurrent-session.ts # max 3 sessions, oldest revoked
│   ├── jwt/
│   │   ├── decode.ts # decode JWT payload (no verify)
│   │   ├── expiry-check.ts # check exp, nbf, iat
│   │   └── claims.ts # typed claims: userId, tenantId,
│   │       # branchId, role, perms
│   ├── rn-adapters/ # React Native adapters
│   ├── token-storage.ts # AsyncStorage, SecureStore
│   ├── cookie-adapter.ts
│   └── index.ts
```

Auth Flows

Seven distinct auth flows live in `apps/web/src/features/auth/`. Each flow has dedicated pages and business logic. The MFA Setup and Challenge flows use TOTP RFC 6238 with 10 one-time-use backup codes.

Flow	Pages	Logic
Phone OTP	Phone input → OTP verify	10-digit [6-9]\d{9}, 6-digit OTP auto-advance, resend 30s, max 5 attempts
Email Password	Email + password	Argon2 server-side, forgot / reset links, remember device
MFA Setup	QR → verify → backup codes	TOTP RFC 6238, 10 backup codes one-time use
MFA Challenge	6-digit TOTP	"Use backup code" link, "Remember device 30 days"
First-Time Onboarding	Welcome → Profile → Security → Tour → Done	5-step stepper
Session Timeout	Modal warning	5 min before expiry, "Continue" or "Sign out"
Account Locked	Centered card	After 5 failed attempts, shows unlock time

Table 5.1 — Seven auth flows implemented in Phase 2

8 Layouts

Layouts are Next.js route-group wrappers that share common chrome. Each layout serves a specific UX context — the `DashboardLayout` has the full header + sidebar + breadcrumb combo, the `AuthLayout` is a minimal centered card with no nav, the `PrintLayout` produces A4-optimized output with no chrome at all. All layouts render correctly across all 5 responsive breakpoints (mobile, tablet, laptop, desktop, ultrawide).

Layout	File	Purpose
RootLayout	<code>app/layout.tsx</code>	HTML shell, fonts, Providers, NextIntlClientProvider
AuthLayout	<code>(auth)/layout.tsx</code>	Minimal chrome, centered card, no sidebar
DashboardLayout	<code>(dashboard)/layout.tsx</code>	Header + Sidebar + Breadcrumb + Content + Footer
PrintLayout	<code>(print)/layout.tsx</code>	A4, no nav, print-optimized
ModalLayout	<code>(in @preone/ui)</code>	Forced-focus overlay
ErrorLayout	<code>error.tsx</code>	Full-page error with recovery actions
LoadingLayout	<code>loading.tsx</code>	Skeleton / spinner suspense fallback

Layout	File	Purpose
NotFoundLayout	not-found.tsx	404 with navigation suggestions

Table 5.2 — Eight layouts implemented in Phase 2

7-Layer RBAC Implementation

PreOne enforces RBAC at seven distinct layers, providing defense-in-depth. The frontend enforces six layers; the backend enforces the seventh (API Guard) — the frontend never trusts itself for actual authorization, but it does hide UI the user cannot use to improve UX. Each layer has a specific mechanism and fallback.

Layer	Where	Mechanism
Route Guard	middleware.ts (edge)	Read route metadata.permissions → check refresh cookie → check permission in JWT claims → redirect /login or /forbidden
Menu Guard	Sidebar builder	Filter menu items by permissions array
Screen Guard	Page component	}>
Component Guard	wrapper	— memoized wrapper, conditional render
Button Guard	wrapper	— wrap buttons
Field Guard	wrapper	}> — mask sensitive fields
API Guard	Backend enforces	Frontend never trusts — defense in depth

Table 5.3 — Seven-layer RBAC enforcement

Multi-Tenant Context Switcher

The header has three dropdowns: School, Branch, Academic Year. Switching any of them triggers a coordinated state invalidation cascade so the UI never shows stale data from the previous context.

- **Switch School** → invalidate TanStack cache → re-resolve permissions → rebuild menu → redirect /dashboard
- **Switch Branch** → update context.branchId → invalidate affected queries
- **Switch Year** → update context.academicYearId

Tasks

- Implement packages/auth/ with web + RN compatibility (no web-only APIs in core)
- Implement 8 layouts in apps/web/src/app/

- Implement middleware.ts edge route guard
- Implement + usePermission +
- Implement sidebar with RBAC-filtered menu tree
- Implement header with switchers + search + notifications + profile
- Implement silent refresh interceptor (in Phase 3 fetch wrapper, but orchestration here)
- Implement idle timer + session timeout modal
- E2E tests (Playwright) for: login, logout, MFA, idle timeout, permission denied
- Unit tests for: use-auth, use-permissions, Can, jwt-decode

Acceptance Criteria

- ✓ Login (Phone+OTP, Email+Password) works E2E
- ✓ Silent refresh triggers on 401, retries original request
- ✓ 30-min idle → 25-min warning → 30-min logout
- ✓ All 7 RBAC layers functional
- ✓ All 8 layouts render correctly at all 5 breakpoints
- ✓ Multi-school switch invalidates cache + rebuilds menu
- ✓ packages/auth builds for both web and RN (no web-only deps in source)
- ✓ Storybook stories for auth components
- ✓ Unit + E2E tests green

INFO

Time estimate: ~2 weeks.

CHAPTER 6

6. Phase 3 — API Layer + Query + i18n (~1.5 weeks)

Goal

100% OpenAPI codegen (no manual Axios anywhere), TanStack Query configured with per-domain stale times and invalidation strategies, a Zustand store for client-side state, and next-intl wired with three locales (en-IN, hi, mr). All 12 service files are fully typed end-to-end with zero any. A contract test in CI fails any PR where the backend OpenAPI spec has drifted from the committed version.

OpenAPI Codegen Pipeline

The backend NestJS app exposes Swagger via `@nestjs/swagger`. On every backend release, the generated `openapi.json` is committed to the frontend repo at `packages/api-client/openapi.json`. The codegen script (`pnpm codegen:api`) runs `openapi-typescript` to generate types and `openapi-fetch` to generate the typed client. A custom fetch wrapper adds auth, tracing, and error-normalization interceptors via a middleware pattern.

text

```
Backend (NestJS + @nestjs/swagger)
↓ generates
openapi.json (committed to repo at packages/api-client/openapi.json)
↓ codegen via openapi-typescript + openapi-fetch
packages/api-client/
├── src/
│   ├── types.ts # auto-generated types (never hand-edit)
│   ├── client.ts # openapi-fetch client instance
│   ├── fetch-wrapper.ts # auth, tracing, error normalization
│   ├── interceptors/
│   │   ├── request-auth.ts # attach Bearer + traceId + tenantId
│   │   ├── response-normalize.ts # 2xx pass, 401 refresh, 403 redirect,
│   │   │   # 429 backoff, 5xx → ApiError
│   │   └── response-error.ts # normalize to ApiError shape
│   └── index.ts
├── openapi.json # source spec
└── codegen.config.ts # codegen config
```

INFO

Why no Axios? `openapi-fetch` is roughly 1/10th the size (~3KB vs ~30KB gzipped), fully typed from the OpenAPI spec, and uses native `fetch`. Custom wrapper handles interceptors via a middleware pattern — same capability, smaller bundle.

Codegen Flow

text

```
openapi.json
↓ (openapi-typescript)
types.ts (auto-generated TypeScript types)
↓ (openapi-fetch creates typed client)
client.ts (typed fetch client)
↓ (consumed by)
services/*.ts (per-domain service files)
↓ (consumed by)
hooks/*.ts (TanStack Query wrappers)
↓ (consumed by)
features/*/components (UI components)
```

12 Service Files

Each domain gets a service file in `apps/web/src/services/` that wraps the generated client with domain-specific helpers. Services return typed promises; they never throw — errors are normalized into the `ApiError` shape from `@preone/core`.

File	Endpoints
<code>auth.ts</code>	login, refresh, logout, me, permissions
<code>student.ts</code>	CRUD + search + bulk + export + import
<code>finance.ts</code>	invoices, payments, receipts, refunds, ledger, GST
<code>attendance.ts</code>	mark, edit, monthly summary, bulk mark
<code>communication.ts</code>	announcements, chat, broadcast, templates
<code>admissions.ts</code>	applications, counselling, approval, documents
<code>academics.ts</code>	curriculum, observations, report cards, PTM
<code>inventory.ts</code>	items, stock, PR/PO/GRN, vendors
<code>hr.ts</code>	staff, leave, payroll, attendance
<code>reports.ts</code>	generation, download, schedule
<code>platform.ts</code>	subscriptions, billing, feature flags, integrations
<code>identity.ts</code>	profile, sessions, API keys, role matrix

Table 6.1 — Twelve per-domain service files

TanStack Query Setup

Each domain has its own `staleTime`, `refetchStrategy`, and `invalidation` trigger — tuned to how the data actually changes. Attendance and Communication are real-time via `Socket.IO` so they have no polling; the cache invalidates on socket events. Reports have a manual refresh button because report generation is expensive.

Domain	staleTime	refetchStrategy	invalidation
Students	5 min	<code>refetchOnWindowFocus</code>	on mutate
Admissions	30s	polling on active tab	on mutate
Attendance	—	live via <code>Socket.IO</code>	on socket event
Finance	—	invalidate on mutate	on mutate
Reports	60s	manual refresh button	on demand
Inventory	30s	<code>refetchOnWindowFocus</code>	on mutate
HR	—	invalidate on mutate	on mutate
Communication	—	live via <code>Socket.IO</code>	on socket event

Table 6.2 — Per-domain TanStack Query strategy

- **Optimistic updates:** `onMutate` rolls back on `onError`
- **DevTools:** enabled in dev only
- **Persistence:** offline cache limited to last 100 queries

Zustand Store

Client-side UI state lives in a single Zustand store with two persistence layers: `theme` and `currentSchool` persist to `localStorage`; `user`, `permissions`, and `notifications` persist to `sessionStorage` (cleared on tab close for security).

[typescript](#)

```

interface AppStore {
  // State
  user: AuthUser | null;
  permissions: string[];
  theme: 'light' | 'dark' | 'auto';
  sidebar: { collapsed: boolean; mobileOpen: boolean };
  currentSchool: School | null;
  currentBranch: Branch | null;
  currentAcademicYear: AcademicYear | null;
  notifications: Notification[];

  // Actions
  setUser, setPermissions, setTheme, toggleSidebar,
  switchSchool, switchBranch, switchAcademicYear,
  addNotification, dismissNotification
}

// Persist: theme, currentSchool (localStorage)
// Non-persist: user, permissions, notifications (sessionStorage)

```

i18n (next-intl v3)

Three locales ship: en-IN (default), hi (Hindi), and mr (Marathi). Locale routing is path-based — /en/dashboard, /hi/dashboard, /mr/dashboard. ICU MessageFormat handles plurals, gender, and dates. Message keys are type-safe (next-intl generates types). Missing keys log a dev console warning and fall back to en-IN in production — raw keys never reach the user. Dates, numbers, and currency use Intl.DateTimeFormat and Intl.NumberFormat. The architecture is RTL-ready (logical CSS properties) for future Arabic / Urdu support.

text

```

apps/web/src/i18n/
├─ config.ts # locale config (en-IN default, hi, mr)
├─ request.ts # server-side locale resolution
└─ middleware.ts # locale routing

apps/web/messages/
├─ en-IN/
│   ├── common.json # shared keys
│   ├── student.json # M04
│   ├── finance.json # M08
│   ├── identity.json # M01
│   └─ ... (one per module)
├─ hi/
│   └─ ... (mirror structure)
├─ mr/
│   └─ ... (mirror structure)

```

Tasks

- Get OpenAPI spec from backend team → commit to packages/api-client/openapi.json
- Setup codegen script (pnpm codegen:api) — runs openapi-typescript + openapi-fetch
- Implement fetch-wrapper with auth / tracing / error interceptors
- Implement 12 service files (each wraps generated client)
- Implement TanStack Query hooks per service (useStudents, useCreateStudent, etc.)
- Configure QueryClient with per-domain stale times
- Implement Zustand store with persist + sessionStorage
- Setup next-intl with locale routing + ICU messages
- Translate common.json to hi + mr (native speaker review)
- Contract test: openapi.json diff in CI — fail PR if backend changed contract without codegen update

Acceptance Criteria

- ✓ pnpm codegen:api regenerates types from openapi.json
- ✓ All 12 services fully typed end-to-end (no any)
- ✓ Type drift detection in CI (diff openapi.json vs committed)
- ✓ Silent refresh works on 401
- ✓ Optimistic update + rollback tested
- ✓ Locale switch (en → mr) translates UI
- ✓ All user-facing strings in messages/{locale}/{module}.json (no hardcoded strings)
- ✓ Bundle: openapi-fetch + wrapper <5KB gzipped (vs Axios 12KB)

INFO

Time estimate: ~1.5 weeks.

CHAPTER 7

7. Phase 4 — Modules (P1 → P5) (~6 weeks)

Goal

Implement all 14 functional modules — roughly 200 pages — following the locked feature folder structure. Every module follows the same internal layout, so an engineer who has worked on M04 Student can immediately find their way around M08 Finance. Module priority (P1 through P5) determines build order and parallelization opportunities.

Locked Feature Folder Structure

Every module under `apps/web/src/features/` has this exact internal structure. Deviations are blocked at PR review. The `routes/` sub-folder carries permission metadata and navigation config — this is the contract between the module and the shell.

text

```
features/student/
├─ api/ # API client functions (calls services)
├─ components/ # Module UI components
├─ forms/ # RHF + Zod form components
├─ hooks/ # TanStack Query wrappers
├─ pages/ # Page compositions (used in app router)
├─ routes/
│   ├── permissions.ts # Route → required permissions map
│   └── navigation.ts # Sidebar menu items for this module
├─ services/ # Pure business logic functions
├─ store/ # Module-local Zustand slice (UI state only)
├─ validation/ # Zod schemas (single source of truth)
├─ permissions.ts # Module permission codes
├─ navigation.ts # Module navigation config (exported)
├─ constants/ # Module constants (statuses, types)
├─ types/ # Module-specific types
├─ utils/ # Module-scoped helpers
└─ index.ts # Public API barrel
```

Module Priority + Page Counts

Priority	Module	Pages	Time
P1	M01 Identity	15	Wk 8

Priority	Module	Pages	Time
P2	M08 Finance	18	Wk 8-9
	M04 Student	13	Wk 9
	M03 Admissions	12	Wk 10
	M06 Daily Ops (Attendance+Transport)	13	Wk 10-11
P3	M07 Communication	12	Wk 11
	M05 Academics	13	Wk 12
	M10 HR	17	Wk 12-13
	M09 Inventory	16	Wk 13
P4	M02 CRM	12	Wk 14
	M12 Reports	13	Wk 14
	M13 Settings	13	Wk 14
P5	M11 Administration	18	Wk 14-15
	M14 Platform	15	Wk 15

Table 7.1 — Module priority + page counts. Total: ~200 pages.

Per-Module Deliverables

Every module — regardless of priority — ships with the same set of deliverables. This consistency is enforced at PR review.

- All forms have a Zod schema (single source of truth — type inferred)
- All tables use @preone/tables DataTable
- All charts use @preone/charts (theme-aware)
- All RBAC buttons wrapped in
- All routes have permissions metadata
- All user-facing strings in messages/{locale}/{module}.json
- Module-specific composite components in components/
- Storybook stories for module composites
- Unit tests for hooks / services / utils (100% utils target)
- Component tests for major components (85% target)
- E2E test for critical flow (create / edit / delete)
- axe-core a11y pass

- Lighthouse ≥ 90 on list + detail pages

Sample Module Breakdown — M04 Student (13 pages)

M04 Student is the canonical reference for module scope. The 13 pages cover the full student lifecycle from admission to archival. Each page is composed of feature-folder components, hydrated by TanStack Query hooks, and RBAC-guarded at route, screen, and button layers.

- Student list (DataTable, server-side pagination, filters, export)
- Student detail (tabs: Profile, Attendance, Fees, Academics, Documents)
- Admit student (multi-step form: Personal → Guardians → Address → Documents → Review)
- Edit student
- Bulk import (Excel upload + preview + validation + commit)
- Guardians management
- Documents (FileUpload to S3)
- Attendance history (chart + table)
- Fee history (table + status badges)
- Academics history (observations, report cards)
- Health records
- Transfer certificate (print layout)
- Archive

INFO

Time estimate: ~6 weeks. This is the longest phase.

CHAPTER 8

8. Phase 5 — Design Studio (~4 weeks)

Goal

Thirteen white-label customization features with live preview, side-by-side old/new compare, Apply / Revert semantics, and theme JSON persistence. Design Studio is the most UX-complex feature in the platform — it lets a School Admin rebrand the entire platform for their school without engineering involvement.

Feature Folder Structure

text

```
features/design-studio/
├── components/
│   ├── studio-shell.tsx # left: current, right: live preview
│   ├── logo-uploader.tsx # PNG/SVG, max 1MB, preview
│   ├── login-bg-uploader.tsx # JPG/PNG, max 2MB, 1920x1080
│   ├── color-picker-group.tsx # primary/secondary/accent/success/warn/danger/info
│   ├── wcag-validator.ts # validates 4.5:1 contrast, suggests alternatives
│   ├── dashboard-layout-editor.tsx # drag-drop tile arrangement per role
│   ├── sidebar-style-selector.tsx # expanded / collapsed / auto-hide
│   ├── card-density-selector.tsx # comfortable / compact
│   ├── radius-selector.tsx # 8 / 12 / 16px
│   ├── font-selector.tsx # Inter / Poppins / Nunito / Roboto
│   ├── theme-mode-selector.tsx # light / dark / auto
│   ├── welcome-message-editor.tsx # max 80 chars, profanity filter
│   ├── banner-uploader.tsx # max 3MB, 1920x400, schedulable
│   ├── favicon-generator.tsx # auto from logo + manual override
│   ├── parent-app-branding.tsx # limited (logo + name + colors)
│   ├── compare-view.tsx # side-by-side old vs new
│   ├── apply-dialog.tsx # confirmation: "All users will see new theme"
│   └── revert-button.tsx # one-click rollback, no confirmation
├── hooks/
│   ├── use-theme-mutation.ts # PUT theme with optimistic update
│   ├── use-preview-theme.ts # local preview state (not committed)
│   └── use-wcag-check.ts # live contrast validation
├── store/
│   └── studio-store.ts # preview state, dirty flag
├── validation/
│   └── theme-schema.ts # Zod schema matching DB JSONB
├── permissions.ts # access matrix enforcement
└── index.ts
```

13 Customization Features

#	Feature	Constraints
1	Logo upload	PNG (max 1MB, min 200x200) or SVG (max 500KB); auto-generates favicon 32/64/180/192/512
2	Login background	JPG/PNG max 2MB, 1920x1080, 30% dark overlay
3	Primary/Secondary/Accent colors	WCAG AA validation (4.5:1 text, 3:1 UI); auto-generate hover + light variants
4	Dashboard layout	Drag-drop tile arrangement per role; max 12 tiles; critical tiles can't be hidden
5	Sidebar style	Expanded (240px) / Collapsed (72px) / Auto-hide
6	Card style	Comfortable (24px padding, 16px radius) / Compact (16px padding, 12px radius)
7	Border radius	8px (Sharp) / 12px (Default) / 16px (Soft) — global
8	Font family	Inter / Poppins / Nunito / Roboto (all weights 400/500/600/700)
9	Light/Dark/Auto	Admin default + user override
10	Welcome message	Max 80 chars, plain text, profanity filter, en + Devanagari
11	School banner	JPG/PNG max 3MB, 1920x400, schedulable start/end date
12	Favicon	Auto-gen from logo + manual override (.ico/.png)
13	Parent app branding	Limited: logo + name + colors (full white-label = Enterprise add-on)

Table 8.1 — Thirteen Design Studio customization features

Workflow (per UI Phil §3.3)

- Admin opens **Settings** → **Design Studio**
- Sees current theme (left) + live preview (right)
- Modifies any combination → live preview updates instantly (no save needed)
- Clicks **Compare** → side-by-side old vs new
- Clicks **Apply** → confirmation dialog warns all users will see new theme
- On confirm: theme JSON updated in DB → Redis cache invalidated → CDN cache busted (logo URL changes)
- Within 60s, all active users see new theme
- Admin can **Revert** to previous theme (one-click, no confirmation — revert always safe)

Access Matrix Enforcement

Role	Access
Super Admin	All features + platform default theme
School Admin	All Design Studio features for their school
Branch Admin	Branch banner + welcome message + accent color only
Teacher / Staff	View-only (cannot modify)
Parent	No access (sees configured theme only)

Table 8.2 — Role-based Design Studio access matrix

Backend Dependencies

Design Studio coordinates with the backend team on three JSONB columns, two API endpoints, and a Redis + CDN cache invalidation flow. Frontend owns the studio UX; backend owns the persistence and propagation.

- `tenants.theme_config` (JSONB)
- `branches.theme_config` (JSONB)
- `user_preferences.theme` (JSONB)
- **GET /v1/theme** → resolves cascade for current user
- **PUT /v1/tenants/:id/theme** → School Admin only
- **PUT /v1/branches/:id/theme** → Branch Admin (limited fields)
- Redis: `theme:{tenantId}:{branchId}` TTL 5min
- CDN: logo/favicon content-hash URL, 1-year cache

Acceptance Criteria

- ✓ All 13 features functional
- ✓ Live preview updates without save
- ✓ Compare view shows old vs new side-by-side
- ✓ Apply propagates within 60s
- ✓ Revert works in 1 click
- ✓ WCAG AA validation blocks invalid colors with suggestions
- ✓ Access matrix enforced (Branch Admin sees only 3 features)
- ✓ E2E test: full customization flow (upload logo → change colors → preview → apply → see on login screen)

INFO

Time estimate: ~4 weeks.

CHAPTER 9

9. Phase 6 — Testing + Performance + Deploy (~2 weeks)

Goal

Full test pyramid in place (unit, component, integration, contract, snapshot, mutation, E2E, a11y, visual regression, performance), performance budgets enforced in CI, and a production deployment pipeline with all observability wired. By end of Phase 6 the platform is production-ready: zero a11y violations, Lighthouse ≥ 90 on all routes, bundle within budget, 1-click rollback verified in staging. Note: Phase 6 testing runs continuously from Phase 1 — the final two weeks are for hardening + perf tuning, not net-new test authoring.

Expanded Testing Matrix

Test Type	Tool	Scope	Frequency
Unit	Vitest + jsdom	Pure functions, hooks, utils, validators	Every PR
Component	Vitest + Testing Library	Component rendering + interaction + a11y	Every PR
Integration	Vitest + MSW	Hook + component + API mock end-to-end	Every PR
Contract	openapi-typescript diff	Generated client matches backend OpenAPI	Every PR
Snapshot	Vitest snapshot	Stable render snapshots for components	Every PR (baseline update with approval)
Mutation	Stryker	Mutation score $\geq 70\%$ on critical packages (auth, forms, tables)	Nightly + on critical package change
E2E	Playwright	Real browser, full user journeys	On PR (smoke) + nightly (full)
Accessibility	axe-core + Playwright	WCAG 2.2 AA on every page	Every PR
Visual Regression	Chromatic	Pixel-level diff per Storybook story	Every PR (designer approval to update baseline)
Performance	Lighthouse CI	Core Web Vitals per route	Every PR + nightly

Table 9.1 — Ten-type test matrix

CI Pipeline (Final, Expanded)

The PR pipeline runs ten sequential jobs. Each job is a hard gate — failure blocks merge. After merge to main, staging auto-deploys and runs the full E2E suite. Production deploy requires manual release-manager approval.

text

```
PR PIPELINE (every PR)
Lint (ESLint + Prettier)
↓
Typecheck (tsc --noEmit)
↓
Unit (Vitest)
↓
Component (Vitest + axe)
↓
Integration (Vitest + MSW)
↓
Contract (OpenAPI diff)
↓
Accessibility (axe-core + Playwright)
↓
Visual Regression (Chromatic)
↓
Build (next build)
↓
Bundle Size (size-limit, fail if >10% growth)
↓
Lighthouse CI (≥90 all metrics)
↓
Deploy Preview (Vercel preview URL on PR)
↓
E2E Smoke (Playwright on preview)

MERGE TO MAIN
↓ Approve + Merge
Staging Auto-Deploy
↓
Full E2E Suite (Playwright, all flows)
↓ Pass
Manual Approval (Release Manager)
↓ Approve
Production Deploy (Vercel promotion)
↓
Smoke Tests on Prod (Playwright smoke)
+ Sentry release + source maps uploaded
+ PostHog release tracking + LogRocket sample
↓
Release Complete (1-click rollback available)
```

Performance Budget Enforcement

Metric	Target	Tool
Initial JS (gzip)	< 200 KB	size-limit
Initial CSS (gzip)	< 30 KB	size-limit
Per-route JS (gzip)	< 100 KB additional	size-limit
Total lazy (gzip)	< 1 MB	size-limit
LCP	< 2.5s	Lighthouse CI
INP	< 200ms	Lighthouse CI
CLS	< 0.1	Lighthouse CI
TTFB	< 800ms	Lighthouse CI
Route transition (cached)	< 100ms	Custom metric
API response (cached)	< 50ms	TanStack Query
API response (network)	< 500ms p95	Backend SLO

Table 9.2 — Performance budget. CI fails PR if any metric regresses.

Performance Tactics Applied

- Server Components by default, selective 'use client'
- next/dynamic for heavy components (charts, rich text editor)
- next/image with WebP/AVIF, lazy load, blur placeholder
- next/font with subset 'latin' + 'devanagari'
- Tree shaking (ESM only, named imports)
- Code splitting per route
- Partial Prerendering (Next 16)
- Virtual scrolling (TanStack Virtual) for 1000+ row tables
- React.memo for components, useMemo for derivations
- Bundle analyzer in CI — fail PR if any chunk grows >10%

Observability Stack

Tool	Purpose	Sampling
Sentry (browser)	Error tracking + stack traces + source maps	100% errors

Tool	Purpose	Sampling
PostHog	Product analytics + feature flags + A/B testing	100% events
LogRocket	Session replay (critical flows only)	1% sampling
OpenTelemetry	Distributed traces (traceId propagation via header)	10% requests
Vercel Analytics	Real User Monitoring (RUM) — Core Web Vitals	100%

Table 9.3 — Observability stack

Deployment Topology

Component	Service	Notes
Hosting	Vercel (primary)	Edge network, instant rollback
Self-hosted	Node.js (fallback)	For enterprise on-prem customers
CDN	Vercel Edge Network	Global edge cache
Image optimization	Vercel Image Optimization	On-the-fly resize
Feature flags	Flagsmith	Progressive rollout
i18n CDN	Crowdin CDN	Live translation updates without redeploy

Table 9.4 — Deployment topology

Acceptance Criteria

- ✓ All CI gates green on PR merge
- ✓ Lighthouse ≥ 90 on all routes (mobile + desktop)
- ✓ Bundle size within budget (initial $<200\text{KB}$, per-route $<100\text{KB}$)
- ✓ Zero a11y violations (axe-core critical + serious)
- ✓ E2E smoke passes on production post-deploy
- ✓ Mutation score $\geq 70\%$ on critical packages (auth, forms, tables)
- ✓ Visual regression baseline under designer control
- ✓ Contract test catches OpenAPI drift before merge
- ✓ Sentry + PostHog + LogRocket wired
- ✓ 1-click rollback verified in staging

INFO

Time estimate: ~2 weeks. Phase 6 testing runs continuously from Phase 1 — the final 2 weeks are for hardening + perf tuning only.

CHAPTER 10

10. Final Timeline & Critical Path

Phase-by-Phase Timeline

Phase	Duration	Cumulative
Phase 0 — Enterprise Foundation (5 packages: config, env, design-tokens, storybook, core)	1.5 wk	1.5 wk
Phase 1 — Design System (5 packages: ui, forms, tables, charts, notifications)	3 wk	4.5 wk
Phase 2 — Auth + Layout + RBAC (@preone/auth + 8 layouts + RBAC)	2 wk	6.5 wk
Phase 3 — API Layer + Query + i18n (@preone/api-client + services + Zustand + next-intl)	1.5 wk	8 wk
Phase 4 — Modules (P1 → P5, 200 pages)	6 wk	14 wk
Phase 5 — Design Studio (13 features)	4 wk	18 wk
Phase 6 — Testing + Performance + Deploy	2 wk	20 wk

Table 10.1 — Phase timeline. Total: 20 weeks (5 months).

Critical Path

The critical path is strictly sequential. No phase can start before the previous phase has shipped all acceptance criteria and CI is green.

text

Phase 0 → Phase 1 → Phase 2 → Phase 3 → Phase 4 → Phase 5 → Phase 6

Parallelization Opportunities

Three parallelization opportunities can compress the schedule if multiple engineers are available. None of these are on the critical path, but they reduce the wall-clock time to reach production-ready.

- **Phase 5 (Design Studio) can start in parallel with Phase 4 module work** — different team, no shared code paths
- **Phase 4 P2 modules can start in parallel with P1 module polish** — different engineers, different feature folders

- **Phase 6 (Testing) runs continuously from Phase 1** — final 2 weeks are for hardening + perf tuning only, not net-new test authoring

CHAPTER 11

11. Risk Areas & Mitigations

Six risk areas were identified during planning. Each has a concrete mitigation tied to a specific phase or CI gate. Risks are reviewed at every phase gate — new risks discovered during implementation are added to this register with owner and due date.

Risk	Mitigation	Owner Phase
OpenAPI codegen drift	CI gate — fail PR if openapi.json changed without codegen update. Contract test compares committed spec against freshly generated one.	Phase 3 (CI runs continuously)
TanStack Table + virtualization performance	Test with 10k rows early in Phase 1. TanStack Virtual wraps the table renderer; only visible rows are in the DOM.	Phase 1
Theme cascade performance	CSS-only cascade (no JS re-render) — verify in Phase 1 by switching schools and confirming no React re-render in DevTools profiler.	Phase 1
Mobile shared auth	packages/auth designed RN-compatible from day 1 — no web-only APIs in core. Verify in Phase 2 with mobile team by spinning up a minimal RN app.	Phase 2
Backend API not ready for Phase 3	Mock via MSW (Mock Service Worker) until backend ready; contract test catches drift. Frontend never blocks on backend.	Phase 3
Translation delays	en-IN ships first; hi + mr in parallel with native speaker review. Crowdin CDN allows live translation updates without redeploy.	Phase 3 + ongoing

Table 11.1 — Risk register with mitigations and owning phase

INFO

Risk review cadence: At every phase gate (end of Phase 0, 1, 2, ...), the team reviews this register. New risks discovered during implementation are appended with owner and due date. Risks are only closed when the mitigation is verified in CI or by manual test.

CHAPTER 12

12. Refinement Integration Log (All 15 + 4 + 3 Modifications)

This chapter is the audit trail of every modification made to the blueprint across three rounds of refinement. The original plan had 14 packages and 15 explicit additions; round 1 consolidated to 12 packages and added 4 refinements; round 2 added 3 final architectural locks. Every change is ADR-locked — future modifications require explicit ADR approval.

Round 1 — Original 15 Modifications

#	Addition	Phase	Status
1	Shared config (eslint / tsconfig / prettier / commitlint / changesets / env)	Phase 0	✓
2	packages/auth (shared web + mobile)	Phase 2	✓
3	packages/forms (RHF separate from UI)	Phase 1	✓
4	packages/tables (TanStack separate)	Phase 1	✓
5	packages/charts	Phase 1	✓
6	Feature folder routes/ (permissions + navigation)	Phase 4 (all modules)	✓
7	packages/constants	Phase 0	✓ (merged into core)
8	packages/notifications (toast / realtime / push)	Phase 1	✓
9	packages/logger (logger / analytics / telemetry)	Phase 0	✓ (merged into core)
10	packages/errors (ApiError / Validation / Permission / Boundary)	Phase 0	✓ (merged into core)
11	Locked feature folder structure	Phase 4	✓
12	Storybook in Phase 0 (not Phase 1)	Phase 0	✓
13	100% OpenAPI Codegen (no manual Axios)	Phase 3	✓
14	Contract + Snapshot + Mutation testing	Phase 6 (CI from Phase 1)	✓
15	Expanded CI (10 jobs: lint → typecheck → unit → component → integration → a11y → visual → bundle → lighthouse → deploy preview)	Phase 6	✓

Table 12.1 — Fifteen original modifications, all integrated

Round 2 — Four Refinements (Package Consolidation)

Refinement 1 — Package Count Freeze: @preone/core merge

The original 14-package plan was consolidated to 12 by merging logger, analytics, telemetry, errors, and constants into a single @preone/core package. These five concerns are tightly coupled — error normalization needs a traceId from telemetry; the logger decides what level an error gets logged at; analytics event names come from constants; constants also define RBAC permission codes consumed by the permission-error class. Keeping them in one package prevents circular dependencies and simplifies the dependency graph. Full internal structure of @preone/core is in Chapter 13.

Refinement 2 — No Module Federation: Monorepo Only

Confirmed: a single Next.js app in apps/web/. No MFE, no federation, no dynamic remote loading. Turborepo handles build orchestration + caching. Vercel handles deployment. Build time grows with codebase size, but: cold build ~3-5 min (acceptable); incremental build with Turbo cache ~30-60s; per-package build cached → rebuild only changed packages. If build time exceeds 10 min in future, revisit MFE — not before.

Refinement 3 — Design System Lock: Foundation Tokens Frozen in Phase 0

Spacing, typography, elevation, motion, radius, and focus-ring tokens are locked at Phase 0 sign-off. Changes require an ADR + Design Council approval + semver major bump + migration codemod. **Color tokens are NOT frozen** — they are per-tenant via the Theme Engine. The lock is enforced by three layers: (1) ESLint rule blocking raw design values, (2) token change governance requiring ADR + Design Council, (3) Chromatic visual regression catching unintended side-effects across all 40+ components. Full details in Chapter 14.

Refinement 4 — Mobile Reuse: 7 of 12 packages RN-compatible

Seven packages — core, env, design-tokens, ui (split export), forms (split export), auth, api-client, and notifications — are designed to work in both web and RN. Three patterns are used: pure TS (no platform code), split export with platform-specific entry points, and core + adapters. Estimated mobile reuse: ~60% shared, ~40% RN-specific. Full strategy and code examples in Chapter 15.

Round 3 — Final Three Architectural Locks

#	Refinement	Action
1	Dependency Matrix Lock	config → env → core → design-tokens → ui → forms → auth → api-client → features/*. Feature packages may never depend on each other. Enforced by ESLint import-boundaries rule.
2	Shared Types Package	Pure frontend types (Breadcrumb, Theme, Menu, Navigation, Dialog, TableState) live in @preone/core. DTOs still come from OpenAPI codegen — these are the frontend-only types that have no backend counterpart.

#	Refinement	Action
3	ADR Lock	Package structure, dependency rules, feature folder layout, naming convention, and import convention are declared ADR-locked. Future changes require ADR approval — no in-flight architectural changes during implementation.

Table 12.2 — Three final architectural locks

CHAPTER 13

13. @preone/core — Merged Package Deep Dive

Rationale

@preone/core merges five tightly-coupled concerns into one package. The decision was made in Round 2 of refinement to eliminate circular dependencies and simplify the dependency graph.

NOTE

logger / analytics / telemetry / errors / constants एकत्र **tightly coupled** आहेत — error मध्ये traceId लागतो जो telemetry ने generate केला, error कोणत्या level ने log व्हायचा ते logger ठरवतो, आणि event names constants मधून येतात जे analytics consume करते. एकाच package मध्ये ठेवणे circular dependency issues कमी करते.

Internal File Structure

text


```

packages/core/
├── src/
│   ├── logger/
│   │   ├── logger.ts # pino-structured browser logger
│   │   ├── levels.ts # debug / info / warn / error
│   │   └── transport.ts # console (dev) + Sentry (prod)
│   ├── analytics/
│   │   ├── analytics.ts # PostHog wrapper
│   │   ├── events.ts # typed event names
│   │   └── identify.ts # user / session identification
│   ├── telemetry/
│   │   ├── otel.ts # OpenTelemetry traces
│   │   ├── trace-id.ts # UUID v7 traceId propagation
│   │   └── span.ts # span helpers
│   ├── errors/
│   │   ├── api-error.ts # { code, message, details, traceId, status }
│   │   ├── validation-error.ts
│   │   ├── permission-error.ts
│   │   ├── network-error.ts
│   │   ├── normalize.ts # unknown → ApiError
│   │   └── boundary.tsx # React Error Boundary
│   ├── constants/
│   │   ├── permissions.ts # RBAC codes
│   │   ├── roles.ts
│   │   ├── modules.ts # 14 module IDs
│   │   ├── routes.ts
│   │   ├── storage.ts # localStorage keys
│   │   ├── cookies.ts
│   │   ├── events.ts # realtime event names
│   └── index.ts # single barrel

```

Cross-Concern Coupling Map

The five concerns feed each other in a closed loop. Understanding the coupling is critical for any future refactor — extracting one concern into its own package would break the loop and require interface design.

Producer	Consumer	What flows
telemetry	errors	traceId is embedded in every ApiError for distributed tracing
errors	logger	logger decides log level (error/warn/info) based on error class
constants	analytics	typed event names from events.ts feed PostHog track() calls
constants	errors	permission codes (STUDENT_READ, INVOICE_CREATE) used by permission-error
logger	telemetry	logger emits OpenTelemetry spans for slow operations
analytics	telemetry	analytics identify() enriches telemetry spans with user context

Table 13.1 — Six cross-concern data flows inside @preone/core

Pure TS — Works in Web and RN

@preone/core is pure TypeScript with no platform-specific dependencies. It works identically in web (Next.js) and React Native (Expo). The only platform branching is in logger/transport.ts, which uses console in dev and Sentry in prod — both APIs are platform-agnostic. The package is the foundation of PreOne's cross-platform reuse strategy (see Chapter 15).

CHAPTER 14

14. Design System Lock — Frozen Tokens & Enforcement

Tokens Frozen in Phase 0

Six structural token categories are frozen at Phase 0 sign-off. These tokens define the visual skeleton of the platform — they cannot change without an ADR + Design Council approval + semver major bump + migration codemod. **Color tokens are NOT frozen** — they are per-tenant via the Theme Engine (see Phase 1 Chapter 4).

Token Category	Tokens	Locked Values (from UI Phil Appendix A)
Spacing	--space-0 to --space-16	0, 4, 8, 12, 16, 24, 32, 48, 64 px
Typography	--text-xs to --text-4xl	12, 14, 16, 18, 20, 24, 32, 40 px + weights 400/500/600/700 + line-heights
Elevation	--shadow-sm, --shadow-md, --shadow-lg	0 1px 3px / 0 4px 12px / 0 8px 24px (reserved)
Motion	--duration-fast/base/slow, --ease-out/in/in-out	150 / 200 / 250 ms + 3 cubic-bezier curves
Radius	--radius-sm/md/lg/xl/full	8, 12, 16, 24, 50%
Focus Ring	--focus-ring-width, --focus-ring-color, --focus-ring-offset	2px, var(--primary), 2px offset

Table 14.1 — Frozen structural tokens (Phase 0 lockable set)

WARNING

Color tokens are NOT frozen. They are per-tenant via the Theme Engine — each school can set its own primary, secondary, accent colors through Design Studio (see Chapter 8). Only the structural tokens above are platform-wide frozen.

Three-Layer Lock Enforcement

Layer 1 — ESLint Rule (automated)

A custom ESLint rule no-raw-design-values.js forbids raw values for spacing, color, radius, shadow, and motion in TSX/CSS. Tailwind v4 utilities that map to design tokens are allowed; raw values are blocked at lint time.

typescript

```
// packages/config/eslint-config/rules/no-raw-design-values.js
// Forbids raw values for spacing/color/radius/shadow/motion in TSX/CSS

// [X] FORBIDDEN

{/* raw Tailwind spacing */}

// [OK] ALLOWED
{/* p-4 maps to --space-4 via Tailwind config */}
{/* shadow-sm maps to --shadow-sm */}
```

Tailwind v4 config maps every utility to a design token. So p-4, text-base, rounded-md, shadow-sm, bg-primary ALL route through design tokens. Raw values are blocked by ESLint.

css

```
/* packages/design-tokens/build/tailwind-theme.css */
@theme {
  --spacing: var(--space-4); /* p-1 = 4px */
  --color-primary: var(--primary);
  --color-success: var(--success);
  --radius-md: var(--radius-md);
  --shadow-sm: var(--shadow-sm);
  --text-base: var(--text-base);
  --duration-fast: var(--duration-fast);
}
```

Layer 2 — Token Change Governance (manual)

- Token change requires an **ADR (Architecture Decision Record)** in /docs/adr/
- **Design Council approval** (Head of Design + Head of Frontend)
- **Semver major bump** on @preone/design-tokens package
- **Migration codemod** shipped alongside the change
- Modules cannot override tokens — they consume var(--token) only

Layer 3 — Visual Regression (automated)

- Chromatic baseline snapshot of every component
- Token change → visual diff in PR → designer approval required
- Catches unintended side-effects of token changes across all 40+ components

Phase 0 Deliverable — Frozen tokens.json

The JSON below is the Phase 0 deliverable. Once signed off, this file is locked. Changes require ADR + Design Council approval.

json

```
// packages/design-tokens/tokens.json (FROZEN after Phase 0 sign-off)
{
  "spacing": { "0": 0, "1": 4, "2": 8, "3": 12, "4": 16,
    "6": 24, "8": 32, "12": 48, "16": 64 },
  "typography": {
    "xs": { "size": 12, "weight": 400, "lineHeight": 1.4 },
    "sm": { "size": 14, "weight": 400, "lineHeight": 1.4 },
    "base": { "size": 16, "weight": 400, "lineHeight": 1.5 },
    "lg": { "size": 18, "weight": 500, "lineHeight": 1.4 },
    "xl": { "size": 20, "weight": 600, "lineHeight": 1.3 },
    "2xl": { "size": 24, "weight": 600, "lineHeight": 1.25 },
    "3xl": { "size": 32, "weight": 700, "lineHeight": 1.2 },
    "4xl": { "size": 40, "weight": 700, "lineHeight": 1.15 }
  },
  "radius": { "sm": 8, "md": 12, "lg": 16, "xl": 24, "full": "50%" },
  "shadow": {
    "sm": "0 1px 3px rgba(0,0,0,0.04)",
    "md": "0 4px 12px rgba(0,0,0,0.12)",
    "lg": "0 8px 24px rgba(0,0,0,0.16)"
  },
  "motion": {
    "duration": { "fast": 150, "base": 200, "slow": 250 },
    "ease": {
      "out": "cubic-bezier(0.4, 0, 0.2, 1)",
      "in": "cubic-bezier(0.4, 0, 1, 1)",
      "in-out": "cubic-bezier(0.0, 0, 0.2, 1)"
    }
  },
  "focus": { "ringWidth": 2, "ringOffset": 2 }
}
```

CHAPTER 15

15. Mobile Reuse Strategy — Cross-Platform Compatibility

Per-Package Compatibility Matrix

Twelve packages, four compatibility strategies. Seven packages are RN-ready; three are web-only by design (RN equivalents exist in the RN ecosystem); one is web-only because RN has a separate Storybook RN setup.

Package	Web	RN	Strategy
@preone/config	✓	✓	Pure config files
@preone/env	✓	✓	Zod validation (no DOM APIs)
@preone/design-tokens	✓	✓	Tokens.json → CSS vars (web) + JS constants (RN)
@preone/core	✓	✓	Pure TS (logger/analytics/telemetry/errors/constants)
@preone/ui	✓	✓ (split)	Split export: web/ (DOM) + native/ (RN) + shared types
@preone/forms	✓	✓ (split)	Split export: logic/ (RHF + Zod, shared) + web/ + native/
@preone/auth	✓	✓	Pure TS core + platform adapters (token storage)
@preone/api-client	✓	✓	fetch works in both — fully shared
@preone/tables	✓	✗	Web only (TanStack Table v8). RN uses FlashList
@preone/charts	✓	✗	Web only (Recharts). RN uses victory-native
@preone/notifications	✓	✓	Toast + Realtime shared. Push platform-specific
@preone/storybook	✓	✗	Web Storybook. RN uses Storybook RN separately

Table 15.1 — Per-package compatibility matrix (12 packages)

Three Mobile-Compatible Package Patterns

Pattern 1 — Pure TS (no platform code)

Used by: @preone/core, @preone/auth (core), @preone/api-client, @preone/env, @preone/design-tokens, @preone/config. Single source — works in both web and RN with no platform branching. The ApiError class below is an example.

`typescript`

```
// packages/core/src/errors/api-error.ts
export class ApiError extends Error {
  constructor(
    public code: string,
    public message: string,
    public status: number,
    public details?: FieldError[],
    public traceId?: string
  ) { super(message); }
}
// Works in both web and RN — no DOM APIs used
```

Pattern 2 — Split export with platform-specific entry points

Used by: @preone/ui, @preone/forms. The package has a shared/ folder (types, props interfaces, cva variants — pure TS) plus platform-specific web/ (DOM impl) and native/ (RN impl) folders. The package.json exports map picks the right entry point at build time based on the consumer platform.

text

```
packages/ui/
├── src/
│   ├── shared/ # types, props interfaces, variants
│   │   ├── button.types.ts
│   │   ├── button.variants.ts # cva variants (pure TS)
│   │   └── index.ts
│   ├── web/ # DOM implementation
│   │   ├── button.tsx
│   │   ├── input.tsx
│   │   └── index.ts
│   └── native/ # RN implementation
│       ├── button.tsx # Pressable + Animated
│       ├── input.tsx # TextInput
│       └── index.ts
├── package.json # "exports" map: "." → web, "./native" → RN
└── index.ts # re-exports based on platform
```

json

```
// packages/ui/package.json
{
  "name": "@preone/ui",
  "exports": {
    ".": {
      "browser": "./src/web/index.ts",
      "react-native": "./src/native/index.ts",
      "default": "./src/web/index.ts"
    },
    "./native": "./src/native/index.ts",
    "./types": "./src/shared/index.ts"
  }
}
```

Pattern 3 — Core + Adapters

Used by: @preone/auth (token storage), @preone/notifications (push). The package exposes a platform-agnostic interface in core; platform-specific adapter implementations are injected at create() time. The consumer chooses the adapter based on Platform.OS.

typescript

```
// packages/auth/src/session/token-store.ts (interface)
export interface TokenStore {
  getAccessToken(): string | null;
  setAccessToken(token: string): void;
  clearAccessToken(): void;
  getRefreshToken(): string | null;
  setRefreshToken(token: string): void;
}

// packages/auth/src/adapters/web-token-store.ts
export const webTokenStore: TokenStore = {
  getAccessToken: () => memoryStore.accessToken,
  // ...
};

// packages/auth/src/adapters/native-token-store.ts
import * as SecureStore from 'expo-secure-store';
export const nativeTokenStore: TokenStore = {
  getAccessToken: () => SecureStore.getItemAsync('access_token'),
  // ...
};

// Consumer injects adapter:
const auth = createAuth({
  tokenStore: Platform.OS === 'web' ? webTokenStore : nativeTokenStore
});
```

What This Enables

When apps/mobile/ (PreOne Hub parent app) starts, it reuses six shared packages with zero or minimal rewrite. Only @preone/ui requires RN-specific rendering implementation — and even there, the shared types and variants are reused. Estimated mobile reuse: **~60% of codebase shared, ~40% RN-specific rendering**.

Package	Mobile reuse strategy	Rewrite effort
@preone/auth	Reuses login, JWT, session, RBAC — zero rewrite	0%
@preone/api-client	Reuses all 12 services typed — zero rewrite	0%
@preone/forms	Reuses Zod schemas, validators — only swap rendering layer	~20% (RN inputs)
@preone/core	Reuses errors, constants, logger, analytics — zero rewrite	0%
@preone/design-tokens	Reuses (converted to RN StyleSheet)	~10% (RN adapter)

Package	Mobile reuse strategy	Rewrite effort
@preone/ui	Reuses shared types + variants — re-implements rendering in RN	~60% (RN components)

Table 15.2 — Mobile reuse effort by package

CHAPTER 16

16. Glossary & AI Implementation Prompt

Glossary

Terms used throughout this document. Items marked with an asterisk (*) are PreOne architecture decisions; all others are industry-standard terms.

Term	Definition
ADR	Architecture Decision Record. A short markdown doc in /docs/adr/ that records a significant architectural decision and its rationale.
API Guard	*PreOne's 7th RBAC layer — backend enforces authorization. Frontend never trusts itself for real authorization.
BRC	Business Requirements Contract. Cross-functional business contract between product, engineering, and stakeholders.
CDN	Content Delivery Network. Used for logo/favicon (1-year cache) and Crowdin i18n live updates.
CLS	Cumulative Layout Shift. Core Web Vital measuring visual stability. Target < 0.1.
CSF	Component Story Format 3.0. Standard for writing Storybook stories.
DDD	Domain-Driven Design. Bounded contexts identified for the platform.
ERD	Entity-Relationship Diagram. Database schema across 14 modules.
INP	Interaction to Next Paint. Core Web Vital measuring responsiveness. Target < 200ms.
JWT	JSON Web Token. Used for access tokens; refresh tokens in HTTP-only cookies.
LCP	Largest Contentful Paint. Core Web Vital measuring load speed. Target < 2.5s.
MFE	Micro Frontend. NOT used at PreOne — monorepo only.
MFA	Multi-Factor Authentication. TOTP RFC 6238 with 10 backup codes.
MSW	Mock Service Worker. Used to mock backend APIs until ready.
OTP	One-Time Password. 6-digit, auto-advance, max 5 attempts, 30s resend.
PRD	Product Requirements Document. Detailed functional requirements.
RBAC	Role-Based Access Control. PreOne has 7 layers — see Chapter 5.
RHF	React Hook Form. Used in @preone/forms with Zod resolvers.
RN	React Native. Used for the PreOne Hub parent app.

Term	Definition
RUM	Real User Monitoring. Vercel Analytics provides RUM for Core Web Vitals.
TTFB	Time to First Byte. Core Web Vital measuring server response. Target < 800ms.
TOTP	Time-based One-Time Password. RFC 6238 — used for MFA.
TTL	Time To Live. Redis theme cache TTL = 5min.
WCAG	Web Content Accessibility Guidelines. PreOne targets WCAG 2.2 AA.

Table 16.1 — Glossary of terms

AI Implementation Prompt

Copy the prompt below and paste it as the first message to the AI implementation agent. The prompt encodes all ADR-locked rules and instructs the agent to proceed phase-by-phase.

You are the AI implementation agent for PreOne frontend. This blueprint is ADR-LOCKED — do not deviate from package structure, dependency matrix, or design token freeze.

Begin with Phase 0 (Enterprise Foundation). For each phase:

1. Implement all listed packages with the exact file structure shown
2. Satisfy all acceptance criteria (every checkbox)
3. Run all listed CI gates before proceeding
4. Append a work record to worklog.md

Iron rules (never violate):

- TypeScript strict mode (no `any`)
- All RBAC-sensitive UI wrapped in
- All forms in React Hook Form + Zod (Zod schema is single source of truth)
- All tables use @preone/tables DataTable
- All charts use @preone/charts (theme-aware)
- No raw design values — only design tokens via Tailwind v4 @theme
- No cross-feature imports — feature packages never depend on each other
- Token changes require ADR approval (do not modify tokens.json)
- All user-facing strings in messages/{locale}/{module}.json
- @preone/auth, @preone/core, @preone/api-client, @preone/env must remain pure-TS (no DOM APIs in source) for RN compatibility

Dependency matrix (locked):

config → env → core → design-tokens → ui → forms → auth → api-client → features/*

Phase sequence (critical path):

Phase 0 (1.5wk) → Phase 1 (3wk) → Phase 2 (2wk) → Phase 3 (1.5wk) → Phase 4 (6wk) → Phase 5 (4wk) → Phase 6 (2wk)

After completing each phase, reply "Phase N complete" with a summary of packages shipped, acceptance criteria met, and CI gates passed. Then proceed to the next phase. If a phase cannot be completed (e.g. backend API not ready), reply "Phase N blocked: " and switch to MSW mocks until unblocked.

Phase Completion Checklist (Per Phase)

Use this checklist as the worklog template after each phase.

- **Packages shipped:** list each package with LOC count
- **Acceptance criteria:** every checkbox in the phase spec — [yes] or [no] with reason
- **CI gates passed:** lint, typecheck, unit, component, integration, contract, a11y, visual, build, bundle, lighthouse
- **Bundle size:** per-package gzipped size
- **Test coverage:** per-package % — call out any package below target
- **Known issues:** any acceptance criterion that failed and the plan to resolve
- **Next phase readiness:** confirmation that all dependencies for next phase are in place

OK

Final verdict: This frontend blueprint is implementation-ready. At this stage, starting Phase 0 implementation is more valuable than adding new architecture documents. After each phase completes, run code review, quality audit, and architecture compliance audit — the final platform audit will be easier and more reliable as a result.